

Revisão Profunda — Quântica Comercial + Avaliação do LangNet

Data: 2026-07-28 · **Revisor:** Claude (agindo como o usuário, pela interface + backend) **Projeto avaliado:** Quântica Comercial (`b55ef718-0073-44d4-b279-11df89403e92`) **Modelo que corrige/refina (no ar):** qwen2.5-coder-32b-instruct (LM Studio 192.168.1.115:1234)

Escopo: revisar cada etapa do pipeline (Especificação → Modelo de Dados → Protótipo de Interface → Agentes/Tarefas → Sequência/Petri → Código final), analisar os artefatos até o código, submeter melhorias ao modelo gerando novas versões, com foco em **interface** e **agentes**. Entregável: avaliação da Quântica + avaliação do LangNet + ajustes necessários, **antes de testar MCP**.

Ambiente

Frontend :3000 ✓ · Backend :8000 ✓ · App gerado :3001 ✓ · ws-server :5002 (fora) · LM Studio ✓ (qwen2.5-coder-32b). Entrei na UI como o dono do projeto (token do usuário `teste@teste.com`).

Mapa das etapas (estado no início)

Etapa	Sessão	Estado
Especificação Funcional	<code>fbcb45992</code>	73.263 chars · completed
Modelo de Dados	<code>6f7183e6</code>	v2 (após fix de coerência) · completed
UI Spec / Protótipo	<code>68607a1b</code>	18 telas · draft
Sequência de Tarefas	<code>3a7b5a9b</code>	—
Código gerado	<code>2630fd53</code>	74 arquivos · ws :5002

1. INTERFACE / PROTÓTIPO — o achado central

Percepção do usuário: "a interface está muito fraca, só tem CRUD." **Veredito honesto (revisei as 18 telas do protótipo + os 35 componentes do código gerado):** ❌ **NÃO** é só CRUD. O gerador produz 4 tipos de tela conforme a intenção do caso de uso:

Tipo	Exemplo (Quântica)	Qualidade
Form (criar/editar)	<i>Nova Persona</i> — campos + chips (Canais, Palavras-chave) + Salvar/Cancelar	Boa
Ação agêntica	<i>Gerar Conteúdo</i> — dropdowns, Headlines (chips), Texto gerado, Gerar Novamente + área de resultado	Boa

Tipo	Exemplo (Quântica)	Qualidade
Dashboard	Coleta de Métricas — cards de KPI (Impressões 12.480...) com deltas ▲8%	Boa
Grid editável	Editar Calendário — tabela com date-picker, dropdown, chips, Salvar/Desfazer	Boa

No **código gerado** (35 telas React): 20 CRUD, 12 agent, 2 report, 1 form. As telas CRUD são **CRUD completo** (lista + busca + "N de M", +Novo, Ver/Editar/Excluir **com confirmação Sim/Não**, detalhe, formulário tipado), ligadas ao backend real por `runTask()` → SQL. As telas agent têm "▶ Executar com IA" + painel de resultado; as report têm export CSV.

Fraquezas REAIS da interface (é isto que faz parecer "fraca")

- 1. **Nome do produto genérico** — sidebar "Nome do Produto"/"MeuProduto", nunca "Quântica". [CORRIGIDO nesta revisão]
- 2. **Navegação lateral inconsistente** — cada tela inventava seu próprio menu. [CORRIGIDO nesta revisão]
- 3. **Dados 100% placeholder** — "Persona X", "Pilar Y", "Lorem ipsum". Sem preview com dados reais do banco.
- 4. **16/39 vínculos de dados quebrados** — telas referenciam tabelas/colunas inexistentes (ver §2).
- 5. **UI genérica para um domínio rico** — sem gráficos, sem seletores relacionais (FK vira caixa de texto de ID), sem calendário/kanban apesar do domínio (calendário editorial, métricas, funil de leads). Toda tela CRUD é o mesmo template.
- 6. **Erros de língua** ("Novo Persona" → "Nova Persona").

Conclusão: o problema não é "só CRUD" — é **branding, navegação unificada, dados reais, coerência com o banco e riqueza semântica** (gráficos/seletores). São ajustes de **geração**, não limitação de fundo. (evidência: docs/revisao-quantica/mockups/*.png — 18 telas)

2. MODELO DE DADOS — coerência com as telas

Rodei o validador de coerência (UC ↔ Mockup ↔ Modelo de Dados) sobre a Quântica:

- **16 de 39 vínculos `bindTo` do protótipo apontam para tabelas/colunas inexistentes.** O LLM do protótipo inventou entidades: `afirmacoes_conteudo`, `agendamento_publicacoes`, `usuarios`, `calendario_mensal`, `itens_calendario` (tabelas inexistentes) e `pilares_conteudo.nome` (coluna).
- Causa: nada validava o mockup contra o schema real; e a regra de tipo de tela "entidade no schema ⇒ tabela" era cega (2 telas — editar/aprovar calendário — foram rotuladas `table`).
- **Ações desta revisão:** apliquei a reconciliação para `pilares_conteudo.nome` → **Modelo de Dados v2** (schema + entities atualizados). Os demais 14 são decisão caso-a-caso: alguns são **alucinação** (ex.: `usuarios` deve religar para a tabela real de usuários), outros são **lacuna real** a adicionar. O painel de Coerência agora oferece as duas opções por item.

Fora isso, o schema em si (19 tabelas) está coerente/normalizado; o problema é o **protótipo divergir do schema**, não o schema estar errado.

3. ESPECIFICAÇÃO FUNCIONAL — qualidade

Estrutura muito rica por UC: cabeçalho, Fluxo Principal (ator ↔ sistema), fluxos alternativos, de exceção, auth, LGPD, auditoria, riscos, **wireframe**. Rastreabilidade a FR-XXX obrigatória.

- **Alinhamento fluxo ↔ wireframe:** bom na prática (13/14 ações citadas no fluxo apareciam no wireframe), mas **não era imposto** pelos prompts — era emergente.
- **Ações desta revisão (LangNet):** adicionei regra de **consistência fluxo↔wireframe** na geração e tornei o **refino ciente de interação** (ao mudar UI de um UC, atualiza fluxo + wireframe juntos, mesmos nomes). Validado ao vivo.

Veredito: etapa **sólida**. É a referência de qualidade do pipeline.

4. AGENTES E TAREFAS — como funcionam (de verdade)

Modelo de execução (um único dispatcher WebSocket, `ws-server/websocket_server.py`): o frontend nunca fala com CrewAI direto — toda ação é `runTask(nome, input)` por `ws://...:5002`.

- **Determinístico-primeiro, CrewAI como fallback:** se `adapters.py` define `<task>_deterministic`, roda **Python puro com SQL no MySQL, sem LLM** (todo CRUD + 4 tarefas de escrita). Senão, monta um **Agent + Task + Crew do CrewAI** e chama `crew.kickoff()`.
- **15 agentes CrewAI reais** (role/goal/backstory/tools) e **16 tarefas** 1:1 com agentes (`agents.yaml` / `tasks.yaml`). Tools por nome via `TOOL_REGISTRY` + `mcp_tools.MCP_TOOLS`.
- **Escolha de LLM:** `PRO_LLM` se o agente tem tools, `FLASH_LLM` se é texto puro. 3 provedores (`LLM_PROVIDER`: deepseek/lmstudio/openai).
- **Duas caras:** Cara A = telas de negócio chamando `runTask`; Cara B = executor de Rede de Petri (`MainExecutor.jsx` + `petri-engine/*`) rodando `petri_net.json`, disparando as MESMAS tarefas.

Nuance importante: na prática, os agentes são **pouco acionados** — os 4 "gêmeos determinísticos" fazem o fluxo principal de dados sem LLM; só tarefas sem gêmeo (verificar fatos, classificar comentários, identificar leads, gerar relatórios) realmente chamam `crew.kickoff()`. Para essas, monta um Crew de **um agente só** por chamada, com `allow_delegation=True` mas **sem pares para delegar**.

5. SEQUÊNCIA DE TAREFAS / REDE DE PETRI

A Rede de Petri é a Cara B (executor administrativo). A Sequência de Tarefas (`3a7b5a9b`) alimenta a Petri. Ambas disparam as mesmas 16 tarefas do dispatcher. (Fora do foco desta revisão; sem bloqueios novos além dos herdados de agents/tasks.)

6. CÓDIGO FINAL GERADO — 74 arquivos (sessão `2630fd53`)

Layout: `ws-server/` (CrewAI + WS), `frontend/` (React, 35 telas), `backend/` (FastAPI fino), `db/schema.sql`, `docker-compose.yml`. O app, como gerado, **NÃO** sobe limpo:

#	Severidade	Problema	Correção
1	● CRÍTICO	Tarefa fantasma <code>aprovar_todos_itens</code> — <code>AprovarCalendarioMensal.jsx</code> chama uma task que não existe em <code>tasks.yaml</code> nem como determinística → erro em runtime. Causa: <code>_agent_screen</code> mantém o <code>target</code> inventado pela UI-spec quando <code>_resolve_task_target</code> pontua <2.	Validar todo <code>target</code> de tela contra <code>tasks.yaml</code> u <code>determinísticas</code> na geração; se não existir, tela desabilitada + aviso.

#	Severidade	Problema	Correção
2	● CRÍTICO	<code>editar_persona_alvo_deterministic</code> usa variáveis indefinidas (<code>prob</code> / <code>obj</code> / <code>palavra</code> em loops que ligam <code>problema</code> / <code>objecao</code> / <code>palavra_chave</code>) → <code>NameError</code> , edição faz rollback e retorna erro.	Corrigir os nomes das variáveis no template do adapter de edição.
3	● ALTO	4 tools referenciadas mas ausentes do registry (<code>cms_api_tool</code> , <code>google_calendar_api_tool</code> , <code>instagram_graph_api_tool</code> , <code>linkedin_api_tool</code>) — <code>_resolve_tools</code> descarta em silêncio → publisher/metrics/calendar rodam sem suas integrações.	Emitir wrapper (stub ou real) para toda tool referenciada; falhar geração se nome não resolver.
4	● ALTO	Tools "reais" são mocks — <code>EmbeddingTool</code> retorna <code>[0.1]*384</code> , <code>VectorSearchTool</code> id fixo, <code>Pdf/Csv/Email</code> retornam string e não fazem nada. Fact-check, classificação, lead-scoring, e-mail são não-funcionais .	Implementar de verdade (sentence-transformers/pgvector, reportlab, smtplib) ou marcar tela "simulado".
5	● MÉDIO	Agentes majoritariamente bypassados (gêmeos determinísticos) — backstories viram peso morto; tarefas agênticas montam Crew de 1 agente sem delegação real.	Decidir por tarefa: determinística OU agêntica; podar a metade não usada; montar crew multi-agente onde fizer sentido.
6	● MÉDIO	Sem auth/multi-tenant/pool — cada função abre conexão MySQL nova; papéis (CEO/Operador/Convidado) existem como tabela mas não são aplicados .	Pool compartilhado, token no handshake do WS, aplicar papéis no servidor.
7	● BAIXO	UI genérica p/ domínio rico; FK como caixa de texto.	Gerar selects relacionais das FKs + ao menos 1 dashboard de métricas.

Bottom line: o modelo de runtime é **coerente e engenhoso** (1 dispatcher WS, determinístico- primeiro com fallback CrewAI, YAML-driven, Petri como face admin). O frontend é **CRUD completo + telas agent/report** — não "só CRUD" — porém **genérico e raso**. Mas há **2 crashes garantidos**, 4 tools faltando e 5 integrações stub: a parte "IA" e de ações externas é **cosmética até corrigir**.

7. AVALIAÇÃO DO LANGNET(nossa aplicação) + AJUSTES NECESSÁRIOS

O pipeline do LangNet está **funcional e coerente ponta a ponta**, mas a revisão expôs ajustes priorizados no **gerador** (é lá que se conserta, não nos artefatos da Quântica):

Prioridade 1 — o app gerado não sobe limpo (bloqueadores)

- **A1.** Validar `target` de cada tela contra as tarefas reais na geração (mata a task fantasma). *[gerador]*
- **A2.** Corrigir o template do adapter de edição (variáveis de loop) — mata o `NameError` . *[gerador]*
- **A3.** Emitir wrapper para toda tool referenciada e **falhar a geração** se alguma não resolver. *[gerador]*

Prioridade 2 — coerência protótipo ↔ dados (fraqueza de interface)

- **A4.**  FEITO: validador de coerência + reconciliação propor-e-aprovar (add ao DM / religar).
- **A5.** Aplicar a reconciliação nos 14 binds restantes da Quântica (religar alucinações, adicionar lacunas reais).

- **A6.** Fazer o mockup **não inventar** entidade/coluna: dar ao LLM só o schema real e proibir campos fora dele.

Prioridade 3 — riqueza e polimento da interface (a queixa do usuário)

- **A7.**  FEITO: branding com nome do projeto + navegação unificada.
- **A8.** Gerar **selects relacionais** a partir das FKs (em vez de caixa de ID) e ao menos 1 **dashboard com gráfico**.
- **A9.** Preencher telas com **amostra de dados reais** do banco (preview), não "Persona X".

Prioridade 4 — agentes de verdade

- **A10.** Implementar as tools mock (embedding/vector/pdf/email) ou marcar "simulado" na tela.
- **A11.** Decidir determinístico-vs-agêntico por tarefa e montar crew multi-agente onde há colaboração.

Já entregue nesta revisão (melhorias no LangNet)

- Consistência **fluxo↔wireframe** na Especificação (geração + refino).
- **Branding + navegação unificada** no protótipo.
- **Validador de coerência + reconciliação** (proposto/aprovado) UC↔Mockup↔DM.
- **Amarração Spec↔Protótipo** (editar interação no spec propaga para a tela).

8. AÇÕES EXECUTADAS NESTA REVISÃO (novas versões / commits)

Ação	Resultado
Extração das 18 telas do protótipo (PNG)	<code>docs/revisao-quantica/mockups/</code>
Análise profunda de código+agentes (74 arquivos)	§4 e §6
Reconciliação de coerência <code>pilares_conteudo.nome</code>	Modelo de Dados v2
Branding + nav unificada (gerador) + regeneração	tela regenerada com "Quântica Comercial"; regeneração completa das 18 telas em andamento (nova versão branded)
Consistência fluxo↔wireframe (Especificação)	commit no gerador

Screenshots usados: `docs/revisao-quantica/mockups/` (18 telas v1) + `mockups-v2/` (branded) + `shots/DEMO-branding-quantica.png` .